Context-Aware Dynamic Tool Resolution for Deadline-Constrained Autonomous Agents

Muhammet Ali Öztürk[1] **and Harun Artuner**[1]

[1]**Hacettepe University, Ankara, Turkiye**

Corresponding author:

Muhammet Ali Öztürk[1]

Email address: muhammet_ozturk@hacettepe.edu.tr, artuner@hacettepe.edu.tr

ABSTRACT

Autonomous agents that use external tools face a deadline-constrained orchestration problem: invoking the slower, more capable *Strategic* pipeline can improve decision quality, but it increases latency and variability and raises the risk of missing deadlines. We formalize this setting as a non-preemptive single-server queue with firm deadlines and two execution modes: a fast *Tactical* mode that earns a base utility, and a slower *Strategic* mode that earns the base utility plus a quality bonus. A missed deadline yields zero utility.

We first establish *Firm-Deadline Tool Control (FTC)*, a feasibility-gating policy whose exact monotone threshold structure we prove in estimated backlog, deadline budget, and time-margin parameter ε, including a generalization to arbitrarily many ordered tool tiers. We then introduce *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with event-driven risk adaptation: on a critical event, CADTR applies a priority-dependent ε-shift that tightens the feasibility budget and biases the decision toward the Tactical mode to protect on-time completion.

We evaluate CADTR against two oracle baselines—a Mode-Aware Oracle with exact per-task queue-wait knowledge, and a Myopic CDF Oracle that maximizes expected on-time utility via a Gaussian CDF approximation—both of which use strictly more queue-state information than CADTR. In trace-driven discrete-event simulations calibrated with three LLM backends (Llama 3.1, Qwen 2.5 7B Instruct, Mistral 7B Instruct), under threat-burst scenarios with 25% critical tasks and arrival rate λ swept from 0.03 to 0.19 req/s, the oracles drop up to $\sim 14\%$ of critical tasks, whereas CADTR maintains a practically flat $\sim 3.5\%$ critical miss rate by throttling Strategic-mode usage on critical tasks from 41% to 5% as load increases.

1 INTRODUCTION

Autonomous agents increasingly use external tools—calculators, retrieval engines, code executors, and structured APIs—to compensate for the limitations of pure model inference Yao et al. (2023); Schick et al. (2023). Tool use improves decision quality, but it changes the serving problem: a single inference becomes a multi-stage pipeline with external dependencies, higher latency, and substantially more variance. In mission-critical applications such as real-time monitoring, autonomous operation, and safety-constrained systems, this variability is consequential, because a missed deadline can leave a task without a usable result Kluge et al. (2015).

We study this as a *multi-resolution tool orchestration* problem. Each arriving task is routed to one of two pipelines: a fast *Tactical* mode that earns a base utility at low latency, or a slower *Strategic* mode that earns a higher utility at the cost of higher and more variable service time. A missed deadline yields zero utility. The orchestrator must maximize the utility delivered on time while protecting critical tasks under congestion.

Classical deadline scheduling and admission control assume that a job's service requirement is fixed once admitted. In tool-augmented serving, the orchestrator can instead *shape the service-time distribution* before execution by selecting a tool-permission profile, trading utility for latency in a way that is coupled to queue state. Classical methods are also semantically blind, treating a routine task and a mission-critical task identically. CADTR adds exactly this missing dimension—a context-dependent risk adaptation that tightens the feasibility budget for critical events—which has no analogue in standard admission control or feasibility scheduling.

We first establish *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy that selects between the Strategic and Tactical pipelines based on predicted deadline feasibility. FTC estimates the waiting time implied by residual in-service work and queued backlog, then tests whether the predicted completion time of the Strategic pipeline fits within the request deadline budget. A single time-margin factor ε relaxes or tightens this test, giving an interpretable operating point that trades on-time utility against deadline-miss rate without changing the underlying model or tool stack. We then introduce *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with event-driven risk adaptation. When a critical event arrives, CADTR applies a context-dependent ε-shift that tightens the feasibility budget and biases the policy toward the Tactical mode; for standard events it behaves identically to FTC. This single additive shift, conditioned on event priority, shields critical tasks under threat-burst load where oracle baselines with more queue-state information do not.

We evaluate orchestration policies with discrete-event simulation driven by empirical end-to-end latency traces collected from a tool-augmented LLM agent. For each prompt we execute both modes, yielding paired samples that reduce routing-induced selection bias and enable trace-driven evaluation without a parametric latency model Agrawal et al. (2024a). Because FTC consumes only calibrated mode-wise latency summaries rather than model-specific internals, the framework can be re-instantiated on another tool-augmented stack by recollecting traces under the corresponding fast/slow profiles.

Contributions. This paper makes four contributions. First, we formalize deadline-constrained tool-augmented inference as a non-preemptive single-server queue with firm deadlines and a multi-resolution utility model in which the Tactical mode earns a base utility, the Strategic mode earns the base utility plus a quality bonus, and a missed deadline yields zero utility. Second, we introduce FTC, a backlog-aware feasibility-gating policy, and prove that it has an exact monotone threshold structure in estimated backlog, deadline budget, and time-margin ε (Theorem 4.1), which extends to arbitrarily many ordered tool tiers (Theorem 4.2). Third, we introduce CADTR, which extends FTC with a priority-dependent ε-shift that shields critical tasks under threat-burst load. Fourth, we evaluate CADTR against a Mode-Aware Oracle and a Myopic CDF Oracle—both with strictly more queue-state knowledge—and show that under a nine-point load sweep with 25% critical tasks the oracles drop up to $\sim$14% of critical tasks while CADTR maintains $\sim$3.5%.

The remainder of the paper reviews related work (Section 2), presents the system model (Section 3), develops FTC and CADTR with their formal properties (Section 4), describes the baseline and oracle policies (Section 5), details the trace-driven methodology (Section 6), and reports results (Section 7) before discussion (Section 8) and conclusions.

2 RELATED WORK

Our work sits at the intersection of (i) tool-augmented LLM agents, (ii) SLO/deadline-aware LLM serving and queue management, and (iii) queueing/control foundations for deadlines and overload protection Nie et al. (2024). We briefly review each area and clarify how our approach differs: FTC and its extension CADTR operate as a *systems-layer* policy that selects among a small set of inference *pipelines* (multi-resolution tool profiles) under firm deadlines, rather than optimizing tool invocation *within* a pipeline or optimizing kernel-level serving mechanics.

2.1 Tool-augmented LLM agents and constrained tool use

Tool-augmented agents motivate our abstraction of multiple inference modes with different quality–latency profiles. ReAct popularized interleaving LM reasoning with external actions, establishing a widely used pattern for tool-using LLM systems Yao et al. (2023). Toolformer demonstrated that language models can learn to invoke tools to improve downstream performance Schick et al. (2023). These works primarily study *content-level* decisions—when and how to call tools to solve a task—and typically optimize task success or answer quality Tao (2025); Gujarati et al. (2020). Existing methods reason about correctness; FTC reasons about schedulability.

Our approach is complementary: it assumes that the tool-augmented inference mode improves expected utility but increases latency and variance, and it controls *whether* a request should be admitted to that mode given the current backlog and the request time budget. Beyond this feasibility gate, CADTR adds a *context-dependent* dimension: the safety margin adapts per-event based on criticality, a mechanism absent from existing tool-augmented agent frameworks.

2.2 SLO and deadline-aware LLM serving

A rapidly growing body of systems work targets SLO attainment in LLM serving by optimizing the *serving stack*—batching, token scheduling, KV (key-value)-cache/memory management, routing, and cluster-level placement Wei (2025).

Kernel- and memory-level systems such as vLLM improve throughput and latency via efficient KV-cache memory management Kwon et al. (2023), and token-level schedulers exploit the prefill/decode phase structure through chunking and schedule-aware execution to balance throughput and tail latency Agrawal et al. (2024b). These techniques reduce per-request cost and control *how tokens are scheduled* once a request is admitted, but they do not decide *which inference pipeline* should run under a firm request TTL; FTC instead makes a mode-selection and admission decision *before* execution to cap the latency variance induced by tool use.

At the orchestration level, serving stacks increasingly incorporate explicit queue-management and SLO-driven policies to stabilize latency under interactive workloads Patke et al. (2024); Hong et al. (2025); Chen et al. (2025); Zhang (2025), including admission control and deadline-style reordering for heterogeneous SLOs Tang et al. (2025); Zhong et al. (2024), dynamic rescheduling and migration across instances Sun et al. (2024), architectural disaggregation of prefill and decode Zhong et al. (2024), and distributed serving at scale Wu et al. (2023). As agentic workflows become common, serving systems also optimize end-to-end workflow latency through program- and lifecycle-aware scheduling Zhang et al. (2023); Luo (2025); Wang (2025a,b), and a related line routes across multiple models or cascades to trade cost, quality, and latency Chen et al. (2023); Ong et al. (2024); Wang et al. (2025). FTC is orthogonal to all of these: it *selects the induced workload* by choosing between tool-permission pipelines using deadline-feasibility reasoning over queue state and measured mode latencies, acting as a lightweight *front-door* policy that composes with the schedulers above rather than replacing them.

2.3 Queueing/control foundations: deadlines, abandonment, feasibility, and drift

FTC is motivated by feasibility reasoning in real-time scheduling, where admission and scheduling decisions are expressed in time-budget units relative to deadlines Liu and Layland (1973); López et al. (2004). Our drift–penalty baseline draws on Lyapunov drift and stochastic network optimization, which balance backlog terms against reward/utility-like objectives Neely (2010). Queueing systems with abandonment emphasize overload protection, robust waiting-time estimation, and deadline-like patience constraints Whitt (2006); Ibrahim and Whitt (2009); O. Garnett (2002). Finally, feedback scheduling connects control-theoretic methods to QoS targets such as miss ratio Stankovic et al. (1999); Lu et al. (2002); Hellerstein et al. (2004).

2.4 Positioning

Viewed through this lens, FTC is a feasibility-style admission/mode-selection policy specialized to tool-augmented inference. It converts observable congestion (queue length and residual in-service

work) together with empirically measured mode latencies into a predicted waiting time, and admits tool-augmented inference only when predicted feasible under the request deadline budget. CADTR extends FTC with *event-driven dynamic risk adaptation*: it adjusts the feasibility margin based on event-level semantic context such as critical versus standard priority, a mechanism with no direct analogue in the classical scheduling, routing, or queueing literature reviewed above. This context-aware dimension distinguishes our contribution from purely load-reactive policies and from static optimization approaches that lack per-event risk adaptation.

3 SYSTEM MODEL

We model a tool-augmented LLM orchestration service that selects an inference mode for each arriving request and executes the resulting pipeline on a shared, non-preemptive server. Requests are associated with firm deadlines (TTLs). If a request has not started service before its deadline, it abandons (is dropped). If it starts service before the deadline, it runs to completion, but yields zero utility if it completes after the deadline. This captures interactive settings where late responses provide no value but still consume capacity.

3.1 Arrivals and deadlines

Requests arrive as a Poisson process with rate $\lambda > 0$; let a_k denote the arrival time of request k. Each request has a TTL budget $\delta_k > 0$ and absolute deadline

$$d_k := a_k + \delta_k. \tag{1}$$

We treat $\{\delta_k\}$ as i.i.d. samples from a regime-dependent distribution. The specific deadline regimes and any truncation/clipping used in experiments are defined in Section 6.3.

The Poisson arrival model is a standard approximation for aggregate, user-driven traffic (and remains a useful baseline even under moderate burstiness), and it allows us to isolate the impact of mode selection and queueing delay on deadline misses.

3.2 Queueing dynamics and timing

The system has an infinite-capacity waiting room and a single server. Service is non-preemptive: once a request begins service, it runs to completion. Let b_k denote the service start time of request k (or $b_k = \infty$ if it abandons), and let c_k denote completion time (or $c_k = \infty$ if it abandons). If request k starts service, its completion time is

$$c_k := \begin{cases} b_k + S_k(m_k), & b_k < \infty, \\ \infty, & b_k = \infty, \end{cases} \tag{2}$$

where m_k is the selected mode and $S_k(m_k)$ is the corresponding service time.

We define the waiting time $\omega_k \coloneqq b_k - a_k$ and end-to-end latency:

$$L_k \coloneqq c_k - a_k = \omega_k + S_k(m_k) + T_{\text{route}}, \tag{3}$$

where T_{route} represents the routing computation overhead.

3.3 Worst-Case Blocking Delay under Non-Preemption

Because service is non-preemptive, a newly arriving high-priority (critical) task can be blocked by a task already in service, regardless of its priority class. Let $S(\text{SLOW})$ and $S(\text{FAST})$ denote the service-time random variables, and let $\mathscr{S}_{\text{SLOW}}$ and $\mathscr{S}_{\text{FAST}}$ denote their respective supports. Let $S_{\max}(\text{SLOW}) \coloneqq \sup \mathscr{S}_{\text{SLOW}}$ be the maximum possible service time of a Strategic task. Formally, if request k arrives at time a_k and finds a Strategic task in service with start time $b_{\text{in-service}} \le a_k$, the waiting time ω_k is lower-bounded by the residual service time of the running task. Under severe threat bursts where the queue is congested, the worst-case waiting time $W_{\max}$ is lower-bounded by this residual service time:

$$W_{\max} \ge \max_{m \in \{\text{FAST},\text{SLOW}\}} \{S_k(m) - (a_k - b_{\text{in-service}})\} \tag{4}$$

yielding a worst-case lower bound:

$$W_{\max} \ge S_{\max}(\text{SLOW}) - \varepsilon_{\text{time}} \tag{5}$$

where $\varepsilon_{\text{time}} \to 0$ represents an arrival immediately after service start. Acknowledging this, the maximum possible waiting time is bounded below by the maximum possible residual service time of a SLOW task:

$$W_{\max} \ge \sup S(\text{SLOW}). \tag{6}$$

This non-preemptive blocking delay makes it impossible to guarantee a zero deadline-miss rate under stochastic service times: any critical task with a deadline budget $\delta_k < S(\text{SLOW})$ that arrives immediately after a Strategic task begins service deterministically violates its deadline.

Realism of assumptions. The single-server abstraction captures a shared bottleneck resource such as a constrained GPU/CPU pool or a serialized tool-execution stage; extending the model and policies to explicit multi-server settings is an important direction for future work.

3.4 Inference modes and service times

The orchestrator selects a mode $m_k \in \mathscr{M}$ at the arrival epoch, where $\mathscr{M} = \{\text{FAST}, \text{SLOW}\}$. We refer to these as the *Tactical* and *Strategic* pipelines, respectively. The Strategic (SLOW) mode runs the more capable pipeline, which typically yields higher utility but has larger mean latency and variance. The Tactical (FAST) mode restricts tool use to reduce latency at the cost of lower utility.

Service time under mode m is modeled as a random variable $S(m)$ with mode-specific distribution F_m. In simulation, we implement F_m using empirical, end-to-end latency traces collected under each mode (Section 6), and sample $S_k(m)$ accordingly.

Estimated mean service times For policy computations that require a point estimate of service time, such as a predicted waiting or sojourn time, we use $\hat{s}(m) := \mathbb{E}[S(m)]$ to denote the *estimated mean service time* under mode m. In particular, $\hat{s}(\text{FAST})$ and $\hat{s}(\text{SLOW})$ are the estimated mean service times of the FAST and SLOW pipelines, respectively, computed from the same mode-specific traces (Section 6).[1]

3.5 Multi-resolution utility model

We adopt a firm-deadline utility model for the two modes. The realized utility of request k is

$$\tilde{U}_k := U_k(m_k)\,\mathbf{1}\{c_k \le d_k\}. \tag{7}$$

Requests that abandon ($b_k = \infty$) or complete after the deadline contribute $\tilde{U}_k = 0$.

The mode-specific success utility $U_k(m)$ is

- Tactical mode (FAST): $U_k(\text{FAST}) = u_{\text{base}}$, the base utility;
- Strategic mode (SLOW): $U_k(\text{SLOW}) = u_{\text{base}} + u_{\text{intel}}$, the base utility plus a quality bonus u_{intel};
- Missed deadline: $\tilde{U}_k = 0$.

In experiments we set $u_{\text{base}} = 1.0$ and $u_{\text{intel}} = 0.5$, with a small Gaussian noise term of standard deviation 0.02. This additive structure makes the trade-off between mode quality and timeliness transparent and avoids ad hoc utility distributions.

3.6 Observable state, policies, and objectives

Let X_k denote the observable system state at arrival of request k. Following standard single-server queueing observables, we use

$$\mathsf{X}_k := (R_k, Q_k), \tag{8}$$

where R_k is the residual service time of the job in service (or $R_k = 0$ if the server is idle) and Q_k is the number of jobs waiting in queue (excluding any job in service).

Predicted wait and related quantities For a *candidate* mode choice $c \in \mathscr{M}$ for request k, we write $\widehat{W}_k^c$ to denote the *predicted waiting time* (predicted time from arrival until service start) that request k would experience if mode c were selected at arrival. This predicted wait is a function of the observable state X_k and the service-time estimates $\hat{s}(\cdot)$; concretely, it represents the

[1] In online deployments, $\hat{s}(m)$ can be updated via a running average or EWMA over recent completions; our experiments use offline estimates from collected traces.

backlog-induced delay implied by the current residual work R_k and the Q_k queued jobs. We also use the corresponding *predicted sojourn time* (predicted end-to-end latency) under candidate mode c,

$$\widehat{L}_k^c \;:=\; \widehat{W}_k^c + \hat{s}(c),$$

i.e., predicted waiting plus the estimated mean service time of the selected mode.

A policy π maps the arrival state and request attributes (including δ_k) to a mode choice: $m_k = \pi(\mathsf{X}_k, \delta_k, \ldots)$. We evaluate policies using (i) mean on-time utility $\mathbb{E}[\tilde{U}_k]$ and (ii) deadline-miss rate (DMR), defined as the fraction of requests that do not complete by their deadlines:

$$\text{DMR} := \Pr[c_k > d_k]. \tag{9}$$

Here, DMR includes both (i) abandoned (never started) and (ii) late completion (started but finished late). These metrics reflect the core trade-off in deadline-constrained tool use: higher utility from the Strategic mode versus increased miss probability from longer service and queueing delay.

3.7 Critical and standard task classes

To model mission-critical scenarios with threat bursts, we consider a two-class system with *critical* (high-priority) and *standard* (normal-priority) tasks sharing the same server. Critical tasks arrive as a Bernoulli-thinned fraction of the Poisson stream (25% critical rate in our experiments). The server selects the head-of-line job from the highest non-empty priority class when it becomes idle (FCFS within each class; service remains non-preemptive). Critical tasks receive a utility multiplier of $1.2\times$, reflecting the higher stakes of mission-critical events. We report both per-class and overall metrics in Section 7. This two-class structure is the foundation of the *threat-burst* evaluation, where CADTR's context-dependent ε-shift provides differentiated risk adaptation for critical tasks.

4 FROM FEASIBILITY GATING TO CONTEXT-AWARE DYNAMIC TOOL RESOLUTION

This section develops the policy framework in two stages. We first introduce *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy that selects between the Strategic (SLOW) and Tactical (FAST) tool pipelines based on predicted deadline feasibility. We then extend FTC to *Context-Aware Dynamic Tool Resolution (CADTR)*, which adds event-driven dynamic risk adaptation for mission-critical scenarios.

4.1 Feasibility gate

At arrival of request k, the observable state is $\mathsf{X}_k = (R_k, Q_k)$, where R_k is the residual service time of the job in service (or 0 if idle) and Q_k is the number of waiting jobs. Let δ_k be the deadline budget for request k. FTC admits request k to the Strategic (SLOW) pipeline only if the predicted

completion time fits within the deadline budget (optionally scaled by $(1+\varepsilon)$), i.e., the feasibility test in Eq. (10) holds:

$$\widehat{W}_k + \hat{s}(\text{SLOW}) \le (1+\varepsilon)\,\delta_k, \tag{10}$$

where $\widehat{W}_k$ is the predicted waiting time until service start and $\hat{s}(\text{SLOW})$ is the expected service time of the Strategic pipeline. If (10) holds, FTC selects SLOW (Strategic); otherwise it selects FAST (Tactical).

4.2 Backlog-based waiting-time estimate

We approximate the waiting time using a standard non-preemptive single-server backlog estimator:

$$\widehat{W}_k := \text{EstimateWait}(R_k, Q_k, \hat{s}^{\text{avg}}) \;:=\; R_k + Q_k\,\hat{s}^{\text{avg}}. \tag{11}$$

Here $\hat{s}^{\text{avg}}$ is an estimate of the mean service time of a typical queued job. Because queued jobs may be served in either mode, we model this typical mean as a convex combination:

$$\hat{s}^{\text{avg}} := \rho\,\hat{s}(\text{SLOW}) + (1-\rho)\,\hat{s}(\text{FAST}), \tag{12}$$

where $\rho \in [0,1]$ is a *queue-mix* parameter representing the expected fraction of queued jobs that will be executed in SLOW mode. Substituting (11) into (10), FTC selects SLOW precisely when

$$R_k + Q_k\,\hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \le (1+\varepsilon)\,\delta_k. \tag{13}$$

This explicit affine form is convenient for both implementation and analysis.

The feasibility inequality (13) is a single affine constraint in $(R_k, Q_k, \delta_k, \varepsilon)$. Several structural properties follow immediately from this form.

Observations.

(O1) If $\varepsilon \le -1$, FTC selects FAST for every arrival state, since $(1+\varepsilon)\delta_k \le 0 < \hat{s}(\text{SLOW})$.

(O2) For $\varepsilon > -1$, FTC defines an exact backlog threshold $B^*(\delta_k, \varepsilon) = (1+\varepsilon)\delta_k - \hat{s}(\text{SLOW})$: SLOW is selected iff $B_k := R_k + Q_k\hat{s}^{\text{avg}} \le B^*$.

(O3) Equivalently, for fixed (R_k, δ_k), this yields a queue threshold $q^* = \lfloor B^* / \hat{s}^{\text{avg}} \rfloor$; and for fixed (R_k, Q_k), a deadline threshold $\delta^* = (R_k + Q_k\hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}))/(1+\varepsilon)$.

These observations confirm that FTC is an exact workload-threshold policy. The following theorem formalizes the key structural property—monotone comparative statics—which shows that FTC's behavior is directionally predictable with respect to all system parameters.

Theorem 4.1 (Monotonicity structure of FTC)**.** *Fix $\rho \in [0,1]$, and let*

$$\hat{s}^{\text{avg}} = \rho\,\hat{s}(\text{SLOW}) + (1-\rho)\,\hat{s}(\text{FAST}),$$

with $\hat{s}(\text{FAST}) > 0$ and $\hat{s}(\text{SLOW}) > 0$. For $\varepsilon > -1$, the FTC decision is nonincreasing in R_k and Q_k, and nondecreasing in δ_k and ε. In particular, if FTC selects SLOW *at $(R, Q, \delta, \varepsilon)$ and*

$$R' \le R, \qquad Q' \le Q, \qquad \delta' \ge \delta, \qquad \varepsilon' \ge \varepsilon > -1,$$

then FTC also selects SLOW *at $(R', Q', \delta', \varepsilon')$.*

Proof. Suppose FTC selects SLOW at $(R, Q, \delta, \varepsilon)$, so that $R + Q\,\hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \le (1+\varepsilon)\,\delta$. Take any $(R', Q', \delta', \varepsilon')$ with $R' \le R$, $Q' \le Q$, $\delta' \ge \delta$, $\varepsilon' \ge \varepsilon > -1$. Since $\hat{s}^{\text{avg}} > 0$, the left-hand side is nonincreasing:

$$R' + Q'\,\hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}) \le R + Q\,\hat{s}^{\text{avg}} + \hat{s}(\text{SLOW}).$$

Since $\delta' > 0$ and $\varepsilon' \ge \varepsilon$, the right-hand side is nondecreasing: $(1+\varepsilon')\delta' \ge (1+\varepsilon)\delta' \ge (1+\varepsilon)\delta$. Combining gives the desired inequality. □

As residual work or queue length grows, the Strategic-tool admission region contracts monotonically; as deadline budget or the margin parameter increases, it expands monotonically. This gives FTC a clean analytical identity and clarifies why it is a stronger comparator than a pure bang-bang queue threshold: FTC switches in deadline units and depends jointly on backlog and request time budget.

The binary fast/slow abstraction is the simplest nontrivial instance of deadline-aware mode gating, but the same feasibility logic is not tied to two modes. In many tool-augmented serving systems, an orchestrator may have access to a finite ordered menu of execution profiles that trade latency for capability more gradually, such as increasingly tool-augmented or increasingly deliberate pipelines. The next theorem shows that the structural threshold property of FTC extends directly to this setting. The only change is that the scalar queue-mix parameter ρ is replaced by a probability vector over modes, which induces a backlog estimate through the corresponding weighted average service-time summary.

Multi-mode extension

Theorem 4.2 (Multi-mode extension of FTC)**.** *Fix $J \ge 2$ modes $\mathscr{M} = \{1, \dots, J\}$, indexed so that*

$$0 < \hat{s}_1 \le \hat{s}_2 \le \cdots \le \hat{s}_J,$$

where mode 1 is the fastest/lightest mode and mode J is the slowest/richest mode. Let

$$\rho = (\rho_1, \dots, \rho_J), \qquad \rho_j \ge 0, \quad \sum_{j=1}^{J} \rho_j = 1,$$

and define the average queued-job service-time estimate

$$\hat{s}_{avg} := \sum_{j=1}^{J} \rho_j \hat{s}_j.$$

For an arrival with observable state (R_k, Q_k) satisfying $R_k \geq 0$ and $Q_k \in \mathbb{Z}_{\geq 0}$, TTL $\delta_k > 0$, and margin $\varepsilon \in \mathbb{R}$, define

$$B_k := R_k + Q_k \hat{s}_{avg}.$$

Consider the J-mode FTC rule that selects the richest feasible mode:

$$m_k^\star := \begin{cases} \max\{j \in \mathscr{M} : B_k + \hat{s}_j \leq (1+\varepsilon)\delta_k\}, & \text{if this set is nonempty,} \\ 1, & \text{otherwise.} \end{cases}$$

Then the following hold.

(i) *If $\varepsilon \leq -1$, then the feasible set is empty for every arrival, and hence $m_k^\star = 1$ for every arrival.*

(ii) *If $\varepsilon > -1$, define for each mode j*

$$B_j^\star(\delta_k, \varepsilon) := (1+\varepsilon)\delta_k - \hat{s}_j.$$

Then

$$B_1^\star(\delta_k, \varepsilon) \geq B_2^\star(\delta_k, \varepsilon) \geq \cdots \geq B_J^\star(\delta_k, \varepsilon),$$

and, for arrivals with a nonempty feasible set, the selected mode is exactly tier-threshold based in estimated backlog:

$$m_k^\star = j \iff B_k \leq B_j^\star(\delta_k, \varepsilon) \text{ and } \big(j = J \text{ or } B_k > B_{j+1}^\star(\delta_k, \varepsilon)\big).$$

If the feasible set is empty, the policy returns mode 1 by convention.

(iii) *Equivalently, for fixed (R_k, Q_k) and $\varepsilon > -1$, define*

$$\delta_j^\star(R_k, Q_k; \varepsilon) := \frac{R_k + Q_k \hat{s}_{avg} + \hat{s}_j}{1+\varepsilon}, \qquad j = 1, \ldots, J.$$

Then

$$\delta_1^\star(R_k, Q_k; \varepsilon) \leq \delta_2^\star(R_k, Q_k; \varepsilon) \leq \cdots \leq \delta_J^\star(R_k, Q_k; \varepsilon),$$

and, whenever at least one threshold is met, $m_k^\star$ is the largest index j such that

$$\delta_k \geq \delta_j^\star(R_k, Q_k; \varepsilon).$$

If no threshold is met, the policy returns mode 1 by convention.

(iv) *On the regime $\varepsilon > -1$, the selected mode is nonincreasing in R_k and Q_k, and nondecreasing in δ_k and ε.*

For $J = 2$, with $(\rho_1, \rho_2) = (1-\rho, \rho)$ and $(\hat{s}_1, \hat{s}_2) = \big(\hat{s}(\text{FAST}), \hat{s}(\text{SLOW})\big)$, this reduces to the two-mode FTC rule in Eqs. (12)–(13) *and Theorem 4.1.*

Proof. Because each $\hat{s}_j > 0$ and ρ is a probability vector,

$$\hat{s}_{avg} = \sum_{j=1}^{J} \rho_j \hat{s}_j > 0.$$

Hence, for any mode j, feasibility of choosing mode j is equivalent to the affine inequality

$$R_k + Q_k \hat{s}_{avg} + \hat{s}_j \le (1+\varepsilon)\delta_k,$$

or equivalently

$$B_k + \hat{s}_j \le (1+\varepsilon)\delta_k.$$

For item (i), if $\varepsilon \le -1$, then $(1+\varepsilon)\delta_k \le 0$ because $\delta_k > 0$. Moreover, since $R_k \ge 0$, $Q_k \ge 0$, and $\hat{s}_{avg} > 0$,

$$B_k = R_k + Q_k \hat{s}_{avg} \ge 0,$$

and therefore

$$B_k + \hat{s}_j \ge \hat{s}_j > 0$$

for every j. Therefore no mode satisfies the feasibility inequality, so the feasible set is empty for every arrival. By definition, the policy then returns mode 1.

For item (ii), assume $\varepsilon > -1$. Rearranging the feasibility condition for mode j gives

$$B_k \le (1+\varepsilon)\delta_k - \hat{s}_j = B_j^\star(\delta_k, \varepsilon).$$

Since

$$\hat{s}_1 \le \hat{s}_2 \le \cdots \le \hat{s}_J,$$

it follows immediately that

$$B_1^\star(\delta_k, \varepsilon) \ge B_2^\star(\delta_k, \varepsilon) \ge \cdots \ge B_J^\star(\delta_k, \varepsilon).$$

Thus, if some mode j is feasible, then every faster mode $1, \ldots, j-1$ is also feasible. Hence the feasible set is either empty or of the form

$$\{1, 2, \ldots, j\}$$

for some j. Since the policy chooses the richest feasible mode, it selects exactly the largest feasible index, which proves the stated characterization for the nonempty-feasible-set case. If the feasible set is empty, the policy definition gives $m_k^\star = 1$.

For item (iii), because $\varepsilon > -1$, one has $1+\varepsilon > 0$, so dividing the same feasibility inequality by $1+\varepsilon$ is valid. Mode j is feasible if and only if

$$\delta_k \ge \frac{R_k + Q_k \hat{s}_{avg} + \hat{s}_j}{1+\varepsilon} = \delta_j^\star(R_k, Q_k; \varepsilon).$$

Since the $\hat{s}_j$ are ordered increasingly, the thresholds $\delta_j^\star(R_k, Q_k; \varepsilon)$ are also ordered increasingly. Therefore, whenever at least one threshold is met, the selected mode is the largest index j whose TTL threshold is met. If no threshold is met, the policy definition gives $m_k^\star = 1$.

For item (iv), if R_k increases or Q_k increases, then

$$B_k = R_k + Q_k \hat{s}_{avg}$$

increases because $\hat{s}_{avg} > 0$, so the set of feasible modes can only shrink; therefore the selected index cannot increase. If δ_k increases or ε increases, then every threshold $B_j^\star(\delta_k, \varepsilon) = (1+\varepsilon)\delta_k - \hat{s}_j$ weakly increases, so the feasible set can only expand; therefore the selected index cannot decrease.

Finally, when $J = 2$, setting

$$(\hat{s}_1, \hat{s}_2) = \big(\hat{s}(\text{FAST}), \hat{s}(\text{SLOW})\big), \qquad (\rho_1, \rho_2) = (1-\rho, \rho),$$

yields

$$\hat{s}_{avg} = (1-\rho)\hat{s}(\text{FAST}) + \rho \hat{s}(\text{SLOW}),$$

which is exactly Eq. (12), and the richest-feasible selection reduces to: choose SLOW if feasible, otherwise choose FAST, which is exactly the current two-mode FTC rule. □

Theorem 4.2 shows that the threshold structure of FTC is not an artifact of the binary fast/slow abstraction. Once the available execution profiles are ordered by their service-time summaries, the same single-server deadline-feasibility argument yields a tiered threshold rule: tighter backlog or smaller TTL pushes the decision toward faster modes, while looser conditions admit progressively richer modes. Thus the current two-mode FTC should be viewed as the minimal special case of a broader finite-mode feasibility-gating family. This extension does not claim new optimality results; rather, it shows that the analytical identity proved in Theorem 4.1 persists when the mode menu is enlarged, with the queue-mix scalar replaced by a simplex-valued calibration parameter.

4.3 Oracle threshold benchmark and relationship to bang-bang control

The threshold form in Theorem 4.1 is a structural property of FTC itself. It is also useful to compare FTC with (i) a one-step oracle that optimizes expected on-time utility directly in slack space and (ii) the bang-bang feasibility controller F-BB. Unlike Theorem 4.1, the oracle result below is not implied by the queueing model alone: its single-crossing condition is an additional benchmark assumption on the mode-wise utility/CDF comparison.

Let $z = \delta - w$ denote residual slack when the actual waiting time is w. For $m \in \{\text{FAST}, \text{SLOW}\}$, let $\bar{u}_m = \mathbb{E}[U(m)]$ and

$$G_m(z) := \bar{u}_m F_m(z), \qquad F_m(z) := \Pr\big(S(m) \le z\big),$$

with $F_m(z) = 0$ for $z < 0$. Then $G_m(z)$ is the myopic expected on-time utility if mode m is chosen at slack z.

Theorem 4.3 (One-step oracle slack threshold)**.** *Assume the difference*

$$D(z) := G_{\text{SLOW}}(z) - G_{\text{FAST}}(z) = \bar{u}_{\text{SLOW}} F_{\text{SLOW}}(z) - \bar{u}_{\text{FAST}} F_{\text{FAST}}(z)$$

has the single-crossing property: there exists $z^\star \in \mathbb{R} \cup \{\pm\infty\}$ such that $D(z) < 0$ for $z < z^\star$ and $D(z) \geq 0$ for $z \geq z^\star$. Then the myopic utility-maximizing action is threshold-based in residual slack:

$$m^\star(z) = \begin{cases} \text{FAST}, & z < z^\star, \\ \text{SLOW}, & z \geq z^\star. \end{cases}$$

Proof. For any fixed slack value z, the myopic expected on-time utility under mode m is exactly $G_m(z)$. Therefore the myopic controller chooses SLOW if and only if $G_{\text{SLOW}}(z) \geq G_{\text{FAST}}(z)$, equivalently if and only if $D(z) \geq 0$. Under the single-crossing assumption, the set $\{z : D(z) \geq 0\}$ is of the form $[z^\star, \infty)$ (allowing the degenerate cases $z^\star = \infty$ and $z^\star = -\infty$). Hence the myopic utility-maximizing action is a threshold rule in residual slack. □

Theorem 4.3 is an oracle benchmark rather than a direct implementation result. It shows that threshold behavior is also compatible with the underlying reward–deadline trade-off, not only with the algebra of FTC's gate. FTC can therefore be interpreted as a lightweight plug-in approximation that replaces the full CDF comparison by a calibrated feasibility test based on backlog summaries.

Proposition 4.4 (F-BB as a special case of FTC)**.** *Setting $\rho = 1$ and $\varepsilon = 0$ in FTC yields*

$$R_k + Q_k \hat{s}(\text{SLOW}) + \hat{s}(\text{SLOW}) \leq \delta_k,$$

which is exactly the F-BB rule in Eq. (16)*.*

Proof. When $\rho = 1$, Eq. (12) gives $\hat{s}^{\text{avg}} = \hat{s}(\text{SLOW})$. Setting $\varepsilon = 0$ in Eq. (13) then yields

$$R_k + Q_k \hat{s}(\text{SLOW}) + \hat{s}(\text{SLOW}) \leq \delta_k,$$

which is Eq. (16). Therefore F-BB is recovered exactly as the special case $(\rho, \varepsilon) = (1, 0)$ of FTC. □

Proposition 4.5 (Slow-admission region nesting and disagreement region)**.** *Fix $\varepsilon = 0$. Let $a = \hat{s}^{\text{avg}}$ and $b = \hat{s}(\text{SLOW})$. Since $\hat{s}(\text{FAST}) \leq \hat{s}(\text{SLOW})$ in the fast/slow ordering and $a = \rho\, \hat{s}(\text{SLOW}) + (1-\rho)\, \hat{s}(\text{FAST})$ is a convex combination of these two values, one has $a \leq b$. Define*

$$\mathscr{S}_{\text{FTC}} := \{(R, Q, \delta) : R + Qa + b \leq \delta\}, \qquad \mathscr{S}_{\text{F-BB}} := \{(R, Q, \delta) : R + Qb + b \leq \delta\}.$$

Then

$$\mathscr{S}_{\text{F-BB}} \subseteq \mathscr{S}_{\text{FTC}}.$$

Moreover, the states in which FTC chooses SLOW *and F-BB chooses* FAST *are exactly*

$$\mathscr{D} := \{(R, Q, \delta) : R + Qa + b \leq \delta < R + Qb + b\}.$$

Proof. Because $a \le b$, one has $R + Qa + b \le R + Qb + b$ for every (R, Q). Therefore any state that satisfies the F-BB feasibility inequality also satisfies the FTC feasibility inequality at $\varepsilon = 0$, which proves the set inclusion. The disagreement region is precisely the set of states in which FTC's inequality holds while F-BB's inequality does not, giving the stated characterization of $\mathscr{D}$. □

Corollary 4.6 (State-dependent crossover under negative margins)**.** *With the notation of Proposition 4.5, FTC requires a larger TTL threshold than F-BB at state (R, Q) if and only if*

$$\frac{R + Qa + b}{1 + \varepsilon} > R + Qb + b,$$

equivalently,

$$\varepsilon < -\frac{Q(b - a)}{R + Qb + b}.$$

Thus negative ε provides an explicit mechanism by which FTC can move from a more aggressive feasibility gate to a more conservative one than F-BB.

Proof. At state (R, Q), FTC requires a TTL of at least $(R + Qa + b)/(1 + \varepsilon)$, whereas F-BB requires a TTL of at least $R + Qb + b$. FTC is more conservative exactly when the former threshold exceeds the latter. Since $R + Qb + b > 0$, rearranging the inequality yields the stated condition on ε. □

4.4 Backlog-estimator refinement and compression error

The queue-mix estimator in Eq. (11) is intentionally lightweight because it needs only (R_k, Q_k) and a scalar ρ. When the scheduler tracks the already assigned modes of queued jobs, a more detailed benchmark is available. Let $n_f(k)$ and $n_s(k)$ denote the numbers of queued FAST and SLOW jobs at the arrival of request k, so that $n_f(k) + n_s(k) = Q_k$. Define the mode-aware benchmark

$$\widehat{W}_k^{\mathrm{mode}} := R_k + n_f(k)\,\hat{s}(\mathrm{FAST}) + n_s(k)\,\hat{s}(\mathrm{SLOW}).$$

Also define the original FTC estimator as

$$\widehat{W}_k^{\rho} := R_k + Q_k\big[\rho\,\hat{s}(\mathrm{SLOW}) + (1 - \rho)\,\hat{s}(\mathrm{FAST})\big].$$

Let

$$\widehat{\rho}_k := \begin{cases} \dfrac{n_s(k)}{Q_k}, & Q_k > 0, \\ 0, & Q_k = 0. \end{cases}$$

Proposition 4.7 (Compression error identity for the queue-mix estimator)**.** *For every arrival k,*

$$\widehat{W}_k^{\rho} - \widehat{W}_k^{\mathrm{mode}} = Q_k\big(\rho - \widehat{\rho}_k\big)\Big(\hat{s}(\mathrm{SLOW}) - \hat{s}(\mathrm{FAST})\Big).$$

Consequently,

$$\big|\widehat{W}_k^{\rho} - \widehat{W}_k^{\mathrm{mode}}\big| \le Q_k\,\big|\rho - \widehat{\rho}_k\big|\,\Big(\hat{s}(\mathrm{SLOW}) - \hat{s}(\mathrm{FAST})\Big).$$

Algorithm 1 FTC (Firm-Deadline Tool Control) at request arrival (defaults: $\rho = 0.55$, $\varepsilon = 0$).

Input: observable state $\mathsf{X}_k = (R_k, Q_k)$; TTL budget δ_k; mode mean service times $\hat{s}(\text{SLOW})$, $\hat{s}(\text{FAST})$; parameters $\rho \in [0,1]$, $\varepsilon \in \mathbf{R}$
Output: selected mode in $\{\text{SLOW}, \text{FAST}\}$
$\hat{s}^{\text{avg}} \leftarrow \rho\, \hat{s}(\text{SLOW}) + (1-\rho)\, \hat{s}(\text{FAST})$
$\widehat{W}_k \leftarrow R_k + Q_k\, \hat{s}^{\text{avg}}$
if $\widehat{W}_k + \hat{s}(\text{SLOW}) \le (1+\varepsilon)\, \delta_k$ **then**
    return SLOW
else
    return FAST
end if

Proof. Because $n_f(k) = Q_k - n_s(k)$, one has

$$\begin{aligned}\widehat{W}_k^{\text{mode}} &= R_k + \big(Q_k - n_s(k)\big)\hat{s}(\text{FAST}) + n_s(k)\hat{s}(\text{SLOW}) \\ &= R_k + Q_k\hat{s}(\text{FAST}) + n_s(k)\Big(\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})\Big).\end{aligned}$$

Similarly,

$$\widehat{W}_k^{\rho} = R_k + Q_k\hat{s}(\text{FAST}) + Q_k\rho\Big(\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})\Big).$$

Subtracting the two identities gives

$$\widehat{W}_k^{\rho} - \widehat{W}_k^{\text{mode}} = \big(Q_k\rho - n_s(k)\big)\Big(\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})\Big) = Q_k\big(\rho - \widehat{\rho}_k\big)\Big(\hat{s}(\text{SLOW}) - \hat{s}(\text{FAST})\Big),$$

where the last step is immediate when $Q_k = 0$ as both sides vanish. Taking absolute values yields the bound. □

Proposition 4.7 clarifies when the lightweight estimator is reliable: the compression error grows linearly with queue length, with the latency gap between the two modes, and with the mismatch between the calibrated ρ and the realized slow-job fraction already sitting in the queue. This also suggests a concrete diagnostic for the empirical section: report the realized queue-mix distribution and compare FTC to a mode-aware variant whenever the scheduler can track queued mode labels.

4.5 Algorithm

Algorithm 1 summarizes the same arrival-epoch decision rule. The theorem above shows that this algorithm implements a monotone threshold policy rather than a purely procedural heuristic. The computation is $O(1)$ per arrival given mode summaries $\hat{s}(\cdot)$.

4.6 Parameterization and operating point

Selecting ρ. In our evaluation, we select ρ via a lightweight grid search over $\rho \in [0,1]$ on a held-out configuration and then fix it for all experiments. Intuitively, smaller ρ makes the backlog estimate

more optimistic (assuming more queued jobs will run in FAST), while larger ρ makes it more conservative. Note that $\rho = 0.55$ is fixed for the remainder of the document unless stated otherwise.

Selecting ε. The time margin ε controls the strictness of the feasibility test in (10). The default $\varepsilon = 0$ corresponds to a strict predicted-feasibility rule. Sweeping $\varepsilon \in \mathbf{R}$ relaxes or tightens the gate and traces a utility–deadline-miss operating frontier. The practically relevant regime is $\varepsilon > -1$; for $\varepsilon \leq -1$, FTC degenerates to ALWAYS-FAST by Theorem 4.1.

FTC* operating point. When we write FTC* in results, it denotes the proposed FTC policy instantiated with its default tuned parameters, namely $\rho = 0.55$ and $\varepsilon = 0$, as determined via the grid-search procedure above.

Mode service-time summaries. FTC requires mode-wise service-time summaries $\hat{s}(\text{FAST})$ and $\hat{s}(\text{SLOW})$. In our experiments, these are computed from empirical latency traces; in deployment, they may be maintained as rolling averages or quantiles over recent observations. The policy itself is not specific to one model identity: its inputs are queue state, deadline budget, and empirical mode-wise latency summaries. Any tool-augmented model/router stack that induces a measurable Tactical/Strategic latency contrast can therefore be calibrated into the same framework by replacing these summaries and replay distributions with traces collected from that stack.

4.7 Context-Aware Dynamic Tool Resolution (CADTR)

FTC treats every task identically: the feasibility gate uses the same ε regardless of task semantics. In mission-critical deployments, certain events carry higher stakes—a critical threat detection, a time-sensitive safety alert, or a high-priority autonomous action—and missing their deadline is especially costly. CADTR extends FTC with event-driven risk adaptation. When a critical-priority event arrives, CADTR applies a context-dependent additive shift $\Delta_\varepsilon^{\text{crit}} < 0$ to the base ε, tightening the feasibility budget and biasing the policy toward the Tactical mode; for standard events it behaves identically to FTC.

For request k with priority class $p_k \in \{\text{critical}, \text{standard}\}$, CADTR selects the effective margin as

$$\varepsilon_k^{\text{eff}} = \begin{cases} \varepsilon + \Delta_\varepsilon^{\text{crit}}, & p_k = \text{critical}, \\ \varepsilon, & p_k = \text{standard}, \end{cases} \tag{14}$$

and then applies the standard FTC feasibility test (Eq. (10)) with $\varepsilon_k^{\text{eff}}$ replacing ε:

$$\widehat{W}_k + \hat{s}(\text{SLOW}) \leq (1 + \varepsilon_k^{\text{eff}})\, \delta_k. \tag{15}$$

A negative $\Delta_\varepsilon^{\text{crit}}$ makes the feasibility gate harder to pass for critical tasks, biasing them toward the Tactical mode and forgoing the quality bonus to improve on-time completion. This is an event-level decision rather than a static parameter: critical tasks are protected from congestion-induced misses, while standard tasks continue to use the Strategic mode when feasible. In our experiments we set

Table 1. Symbols used by CADTR (Algorithm 2).

Symbol	Meaning
R_k	Residual service time of the job in service (0 if idle)
Q_k	Number of jobs waiting in queue
δ_k	Deadline budget (TTL) of request k
p_k	Priority class of request k: critical or standard
$\hat{s}(\text{FAST})$	Mean service time of the Tactical mode
$\hat{s}(\text{SLOW})$	Mean service time of the Strategic mode
ρ	Queue-mix parameter: expected fraction of queued jobs run in SLOW
$\hat{s}^{\text{avg}}$	Mixed mean service time, $\rho\,\hat{s}(\text{SLOW}) + (1-\rho)\,\hat{s}(\text{FAST})$
$\widehat{W}_k$	Predicted waiting time until service start
ε	Base time-margin parameter
$\Delta_\varepsilon^{\text{crit}}$	Critical-event margin shift (< 0)
$\varepsilon_k^{\text{eff}}$	Effective margin applied to request k

$\Delta_\varepsilon^{\text{crit}} = -0.20$, tightening the feasibility window by 20% of the deadline budget for critical events. Table 1 defines the symbols used by the policy and Algorithm 2 gives the procedure.

Because CADTR is FTC with a state-dependent ε substitution, the monotonicity structure of Theorem 4.1 is inherited: within each priority class, CADTR remains a monotone threshold policy in (R_k, Q_k, δ_k). The multi-mode generalization of Theorem 4.2 also extends by substituting $\varepsilon_k^{\text{eff}}$ into the tier-threshold rule.

5 BASELINE POLICIES

We evaluate CADTR against four baseline orchestration policies, deliberately designed to span a hierarchy of information access—from basic feasibility gating (F-BB) to full single-task queue-wait knowledge (Mode-Aware Oracle) and optimal expected-utility maximization (Myopic CDF Oracle). All policies decide at the arrival epoch and share the same queueing dynamics, service-time/utility models, and experimental conditions. All hyperparameters are selected via grid search on a held-out configuration and then fixed for the reported experiments.

5.1 FTC* (tuned FTC baseline)

FTC* denotes the base FTC policy (Algorithm 1) instantiated with its default tuned parameters ($\rho = 0.55$, $\varepsilon = 0$). FTC* treats all tasks identically—it has no priority-dependent adaptation. FTC* represents our own foundational algorithm without the context-aware extension. Serving as an ablation baseline, comparing CADTR to FTC* isolates and quantifies the exact contribution of the dynamic ε-shift.

Algorithm 2 CADTR (Context-Aware Dynamic Tool Resolution) at request arrival.

Input: state $X_k = (R_k, Q_k)$; deadline budget δ_k; priority $p_k \in \{\text{crit}, \text{std}\}$; mode service times $\hat{s}(\text{SLOW}), \hat{s}(\text{FAST})$; base margin ε; critical shift $\Delta_\varepsilon^{\text{crit}}$; queue-mix ρ
Output: selected mode in $\{\text{SLOW}, \text{FAST}\}$
if $p_k = \text{critical}$ **then**
    $\varepsilon_k^{\text{eff}} \leftarrow \varepsilon + \Delta_\varepsilon^{\text{crit}}$
else
    $\varepsilon_k^{\text{eff}} \leftarrow \varepsilon$
end if
$\hat{s}^{\text{avg}} \leftarrow \rho\, \hat{s}(\text{SLOW}) + (1-\rho)\, \hat{s}(\text{FAST})$
$\widehat{W}_k \leftarrow R_k + Q_k\, \hat{s}^{\text{avg}}$
if $\widehat{W}_k + \hat{s}(\text{SLOW}) \le (1 + \varepsilon_k^{\text{eff}})\, \delta_k$ **then**
    return SLOW {Strategic tool}
else
    return FAST {Tactical tool}
end if

5.2 TTL-feasibility bang-bang (F-BB)

F-BB is a conservative feasibility baseline that uses the all-SLOW workload estimate

$$\widetilde{W}_k := R_k + Q_k\, \hat{s}(\text{SLOW})$$

and selects SLOW if and only if

$$\widetilde{W}_k + \hat{s}(\text{SLOW}) \le \delta_k, \tag{16}$$

otherwise it selects FAST. F-BB uses residual work, queue length, and deadline budget, but assumes that every queued job consumes the full Strategic mean service time and exposes neither the queue-mix parameter ρ nor the operating-point parameter ε.

5.3 Mode-Aware Oracle

The Mode-Aware Oracle computes the exact predicted waiting time for each arriving task by inspecting the *actual assigned mode* (Tactical or Strategic) of every task already in the queue, rather than approximating via the queue-mix parameter ρ. For request k with Q_k queued tasks, the waiting time is computed as

$$\widehat{W}_k^{\text{MA}} = R_k + \sum_{j=1}^{Q_k} \hat{s}(m_j), \tag{17}$$

where m_j is the mode already assigned to the j-th queued task. The oracle then applies the same FTC feasibility test using this exact backlog estimate. This baseline controls for the approximation error introduced by the queue-mix parameter ρ: if FTC's aggregate backlog estimate is materially inaccurate, the Mode-Aware Oracle should outperform it. It has *strictly more queue-state information* than CADTR, since it knows every queued task's mode assignment.

Table 2. Information requirements and decision logic of evaluated policies.

Policy	Queue state	Per-task modes	Deadline	Priority
F-BB	(R_k, Q_k)	No	Yes	No
FTC*	(R_k, Q_k)	No	Yes	No
Mode-Aware Oracle	(R_k, Q_k)	Yes	Yes	No
CDF Oracle	(R_k, Q_k)	Yes	Yes	No
CADTR	(R_k, Q_k)	No	Yes	Yes

5.4 Myopic CDF Oracle

The Myopic CDF Oracle implements the single-task expected-utility maximizer derived from the theoretical analysis (related to Theorem 4.2). For each arriving task, it selects the mode m^* that maximizes the expected on-time utility:

$$m_k^* = \arg \max_{m \in \{\text{FAST,SLOW}\}} \hat{u}(m) \cdot \Phi\left(\frac{\delta_k - \widehat{W}_k^{\text{MA}} - \hat{s}(m)}{\hat{\sigma}(m)} \right), \tag{18}$$

where $\hat{u}(m)$ is the expected mode utility, $\widehat{W}_k^{\text{MA}}$ is the Mode-Aware Oracle's exact waiting-time estimate (Eq. 17), $\hat{s}(m)$ and $\hat{\sigma}(m)$ are the empirical mean and standard deviation of service time under mode m, and $\Phi(\cdot)$ is the standard normal CDF.

This oracle combines the Mode-Aware Oracle's exact queue knowledge with a probabilistic completion model, and it greedily maximizes per-task expected utility. It is the strongest conceivable single-task baseline: it uses more information (exact per-task modes) and a more sophisticated decision rule (expected-utility optimization) than CADTR. When this oracle fails under load, it reveals that myopic per-task optimization cannot protect critical tasks—a qualitative insight that motivates CADTR's dynamic risk adaptation.

5.5 Summary of information requirements

Table 2 highlights a key asymmetry: both oracles access strictly more queue-state information than CADTR (exact per-task mode assignments), yet CADTR accesses a different *kind* of information (event priority). As the results in Section 7 demonstrate, the qualitative semantic signal ("is this task critical?") proves more valuable for protecting critical tasks than the quantitative informational advantage of exact queue-wait computation.

6 EXPERIMENTAL SETUP

We evaluate FTC and baselines using discrete-event simulation driven by empirically measured end-to-end latencies for tool-augmented LLM inference. This section specifies the simulator, trace sources, deadline (TTL) regimes, utility generation, hyperparameter selection, and reported metrics.

6.1 Discrete-event simulator

We simulate a non-preemptive single-server queue with an infinite waiting room and firm deadlines as defined in Section 3. Arrivals follow a Poisson process with rate λ. Upon each arrival, the policy selects a mode $m_k \in \{\text{FAST}, \text{SLOW}\}$. Service time $S_k(m_k)$ and success utility $U_k(m_k)$ are sampled as described below. A request yields realized utility $\tilde{U}_k = U_k(m_k)\mathbf{1}\{c_k \le d_k\}$ (Eq. (7)). We report steady-state averages after a warm-up period; all point estimates are averaged across multiple random seeds (see Section 6.8). Routing computation overhead T_{route} is assumed to be statically negligible (< 5 ms) compared to the inference latencies of the LLMs and is therefore excluded from the simulation timing equations.

6.2 Empirical service-time traces

For each mode m, we obtain empirical end-to-end latency samples from a dedicated trace-generation harness and implement the service-time distribution F_m by trace replay (sampling with replacement). Each sample measures the full execution time of the corresponding pipeline, including model generation and any tool overhead present in SLOW. This keeps the simulator nonparametric at the service-time layer and preserves the heavy tails, multi-modality, and mode separation observed in the measured traces.

Counterfactual paired measurements A naive trace-collection protocol can confound prompt difficulty with mode choice: if a router tends to send harder prompts to the tool-augmented pipeline, then mode-wise latency pools mix the effect of the prompt with the effect of the mode. To reduce this selection bias, we additionally collect counterfactual pairs by executing the same prompt under both modes. The resulting paired samples condition on the prompt and isolate the marginal latency effect of enabling the slow tool-augmented pipeline. These pairs are used both to estimate mode-wise service-time summaries and to support controlled offline policy comparisons under a common prompt mix. Unless otherwise noted, the simulation uses the mode-wise empirical distributions derived from these paired measurements.

6.3 Deadline (TTL) regimes

Each request k has a TTL budget δ_k and absolute deadline $d_k = a_k + \delta_k$. Our primary experiments model TTL heterogeneity via a bounded (clipped) normal distribution:

$$\delta_k \sim \text{clip}\big(\mathcal{N}(\mu_\delta, \sigma_\delta^2),\, \delta_{\min},\, \delta_{\max}\big), \tag{19}$$

where $(\mu_\delta, \sigma_\delta)$ are set per regime as given in Table 3 and $(\delta_{\min}, \delta_{\max})$ are (0.05, ∞) respectively for all regimes. We report the regime parameters in Table 3.

Rationale and static-TTL sensitivity Although many interactive systems enforce a nominally fixed timeout, effective time budgets can vary due to tiering, upstream latency, retry budgets, or policy-driven SLO adjustments. This reflects scenarios where user-facing timeouts vary around a

Table 3. Deadline (TTL) regimes used in experiments: $\delta_k \sim \mathcal{N}(\mu_\delta, \sigma_\delta^2)$

Regime	μ_δ (s)	σ_δ (s)
Regime I	28.0	9.0
Regime II	35.0	10.0
Regime III	45.0	12.0
Regime IV	60.0	15.0

mean with some variation. The clipped-normal model provides a controlled way to introduce deadline heterogeneity while holding mean budgets comparable across regimes. To verify that conclusions do not depend on TTL randomness, we also evaluate a static TTL setting:

$$\delta_k \equiv \mu_\delta, \tag{20}$$

and report consistent qualitative ordering of policies under both models.

6.4 Utility generation

The utility model instantiates the multi-resolution framework from Section 3.5. For each request, the mode-specific success utility is drawn from a peaked distribution centered on the physically motivated values:

$$U_k(\text{FAST}) \sim \mathcal{N}(u_{\text{base}}, \sigma_u^2), \quad u_{\text{base}} = 1.0,$$
$$U_k(\text{SLOW}) \sim \mathcal{N}(u_{\text{base}} + u_{\text{intel}}, \sigma_u^2), \quad u_{\text{intel}} = 0.5,$$

with $\sigma_u = 0.02$ providing minimal noise. All values are clipped to $[0, \infty)$. The firm-deadline rule $\tilde{U}_k = U_k(m_k)\mathbf{1}\{c_k \le d_k\}$ (Eq. (7)) applies, so missed deadlines yield $\tilde{U}_k = 0$.

Critical-task utility scaling For the two-class experiments (Section 3.7), critical tasks receive a $1.2\times$ utility multiplier, reflecting the higher operational stakes of mission-critical events. Standard tasks use the base utility values above.

6.5 Policy parameters and hyperparameter selection

FTC and CADTR defaults Unless explicitly varied, FTC uses $\varepsilon = 0$ and $\rho = 0.55$ (selected via grid search on a held-out configuration and fixed for all experiments). CADTR inherits these FTC parameters and additionally uses the critical-event ε-shift $\Delta_\varepsilon^{\text{crit}} = -0.20$.

Baseline parameters The F-BB baseline requires no tunable parameters beyond the mode service-time summaries. Both oracle baselines (Mode-Aware and CDF Oracle) use the same empirical service-time summaries as FTC but compute exact per-task waiting times from actual queue mode assignments.

Lambda sweep Our primary experiment sweeps the arrival rate λ across nine values from 0.03 to 0.19 req/s, with a critical-task fraction of 25%. The deadline budget is a fixed (deterministic) TTL of 35 s (the static-TTL regime of Section 6.3). Results are averaged over 30 replications (seed base 42). Each replication runs to a simulation horizon of 100,000 time units with a 1,000-time-unit warm-up period.

6.6 Trace-generation system and LLM configuration

The trace-generation harness executes a two-tool routing agent on a fixed local compute environment, using an Ollama serving stack with three foundation-model backends: Llama 3.1, Qwen 2.5 7B Instruct, and Mistral 7B Instruct. The agent exposes two execution profiles—a lighter fast profile that requests a shorter answer and a richer slow profile that requests a longer answer—and the harness logs prompt-level metadata, router decisions, error indicators, response lengths, the model-backend identifier, and the paired end-to-end latencies for both modes on the same prompt (Section 6.2). The prompt set is deliberately mixed, spanning short factual or arithmetic requests and longer prompts that require explanation, comparison, planning, or multi-step reasoning. The main queueing experiments replay the Llama 3.1 paired trace as the service-time source; the Qwen and Mistral backends are collected with the same harness and used for the cross-backend service-time comparison of Section 7.3.

The two modes differ in response style as well as latency. Excluding errored rows ($n = 1332$), `fast_lookup` responses are short (mean ≈ 173 characters, median ≈ 156, $p_{90} \approx 333$), whereas `deep_reasoner` responses are substantially longer (mean ≈ 3837, median ≈ 3761, $p_{90} \approx 5343$). This length contrast is the application-level phenomenon that the FAST/SLOW abstraction models, and it drives the calibration of $\hat{s}(\text{FAST})$, $\hat{s}(\text{SLOW})$, and the replay distributions F_m.

Trace collection was run on Windows 11 (64-bit) with an AMD Ryzen 7 5800H CPU, an RTX 3080 Laptop GPU (8 GB), and 31.4 GiB RAM, using Python 3.9.13; these specifications contextualize absolute latencies. We do not claim that absolute latency values transfer unchanged to every deployment, but the queueing conclusions are not tied to one model identity: FTC uses only backlog, TTL, and calibrated mode-wise latency summaries, and the same qualitative fast/slow separation is visible across all three measured backends.

6.7 Metrics

We report: (i) *deadline-miss rate (DMR)* $\Pr[c_k > d_k]$, (ii) *mean on-time utility* $\mathbb{E}[\tilde{U}_k]$, and (iii) latency summary statistics such as p95 end-to-end latency where appropriate. For strict-priority experiments, we additionally report per-class metrics and the resulting class-wise utility–timeliness trade-offs.

6.8 Reproducibility

For each configuration $(\lambda, \text{TTL regime}, \text{policy})$, we run multiple independent simulation replications with different random seeds and report averages across seeds after a warm-up period. All simulator logic, trace processing, and experiment scripts are available in the accompanying repository.

6.9 Data preprocessing

The empirical traces undergo a small, fully deterministic preprocessing pipeline before they are used as the service-time source for the simulator. (i) *Error filtering*: rows corresponding to failed or malformed generations are removed; after filtering, $n = 1332$ paired fast/slow samples remain for the mode-characterization statistics reported in Section 6.6. (ii) *Pairing*: for each prompt and model backend we retain the matched $(S^{\text{fast}}, S^{\text{slow}})$ latencies produced by executing the same prompt under both modes, so that each retained record is a counterfactual pair. (iii) *Service-time source*: the simulator replays the Llama 3.1 paired trace as its service-time distribution, sampling with replacement. (iv) *Summary estimation*: the mode-wise mean service times $\hat{s}(\text{FAST})$ and $\hat{s}(\text{SLOW})$ and the standard deviations $\hat{\sigma}(m)$ used by the policies are computed from this trace. The Qwen 2.5 7B Instruct and Mistral 7B Instruct backends, collected with the same harness, are used for the cross-backend service-time comparison (Section 7.3). No outlier removal, imputation, or distributional smoothing is applied; sampling with replacement preserves the heavy tails, multi-modality, and fast/slow separation present in the measured data.

6.10 Use of AI-assisted tools

The core ideas and scientific contributions of this work—the mathematical formulation, theorems and proofs, system model, experimental design, and all reported results—were conceived and authored by the authors, and the underlying mathematics was specified prior to implementation. AI tools were used solely as assistants under the authors' direction: to translate the author-specified algorithms and experiments into code, and to correct grammar and improve the phrasing of the manuscript LaTeX source. The authors reviewed, verified, and edited all AI-assisted output and take full responsibility for the code, results, and text. The tools used were OpenAI Codex (versions 5.3 and 5.4; `https://openai.com/codex`) and Google Gemini 3.5 Flash, accessed via Google Antigravity (`https://antigravity.google`), and Anthropic Claude Opus 4.8 (`https://www.anthropic.com`), accessed via Anthropic Claude Code. Codex was the primary coding assistant; Claude Opus 4.8 performed the final restructuring of the code repository for release; Gemini 3.5 Flash assisted with porting the manuscript to the journal LaTeX template and with language editing. The complete development history of the code, including its pre- and post-AI-editing states, is publicly available in the project repository (`https://github.com/maliozturk/Deadline_Aware_Tool_Permissioning`), and the coder-agent prompt record is provided as a supplementary file.

7 RESULTS

To isolate the impact of our dynamic risk adaptation, we evaluate CADTR against FTC* (our ablation baseline), alongside F-BB and the two mathematically optimal Oracles. The evaluation is conducted using the trace-driven simulator described in Section 6. The primary experiment sweeps arrival rate λ from 0.03 to 0.19 req/s with 25% critical tasks and a fixed deadline budget of 35 s. All results are averaged over 30 replications. We report per-class and overall metrics: miss rate (DMR), mean realized utility, and Strategic tool (slow-mode) usage fraction.

7.1 Critical-task protection

Figure 1 shows the central finding: CADTR maintains a practically flat $\sim$3.5% critical-task miss rate across the full load sweep, while the oracle baselines—which use strictly more queue-state information—degrade sharply as load increases. At $\lambda = 0.11$, the CDF Oracle drops 14.6% of critical tasks and the Mode-Aware Oracle drops 11.6%, compared to CADTR's 3.4%. At $\lambda = 0.19$, the oracles still miss $\sim$11–14% of critical tasks while CADTR holds at 3.5%. To confirm statistical significance, we perform a paired t-test comparing CADTR against the oracle baselines at the representative medium load $\lambda = 0.11$. The critical miss rate reduction under CADTR is highly statistically significant, yielding $p < 10^{-30}$ against both the Myopic CDF Oracle ($t = -100.64$, $p = 1.88 \times 10^{-38}$) and the Mode-Aware Oracle ($t = -74.51$, $p = 1.11 \times 10^{-34}$).

This result is not explained by informational advantage: CADTR uses *less* queue-state information than either oracle. The difference is CADTR's event-driven ε-shift, which provides *semantic awareness*—the ability to distinguish "this is a critical event that must not fail" from "this is a routine task." The oracles, despite their mathematical sophistication, are blind to event semantics and treat all tasks identically.

Figure 2 shows the mechanism behind this protection: CADTR throttles Strategic-mode usage on critical tasks from 41% at low load ($\lambda = 0.03$) to 5% at high load ($\lambda = 0.19$). The oracle baselines instead maintain $\sim$9% Strategic usage at high load—insufficient throttling, because they do not distinguish critical from standard tasks. As congestion grows, the ε-shift contracts the Strategic admission region for critical tasks, routing them preferentially to the Tactical mode and reducing their exposure to congestion-induced misses.

7.2 Overall utility and miss rate

CADTR's critical-task protection does not come at the expense of system-level utility (Figure 3). Across the load sweep, CADTR and F-BB maintain the highest overall utility, while both oracle baselines lose utility through their elevated miss rates.

The aggregate deadline miss rate across all task classes is shown in Figure 4. F-BB achieves the lowest overall miss rate through its conservative all-SLOW backlog assumption. CADTR achieves the second-lowest overall miss rate, outperforming both oracle baselines across the sweep. The CDF

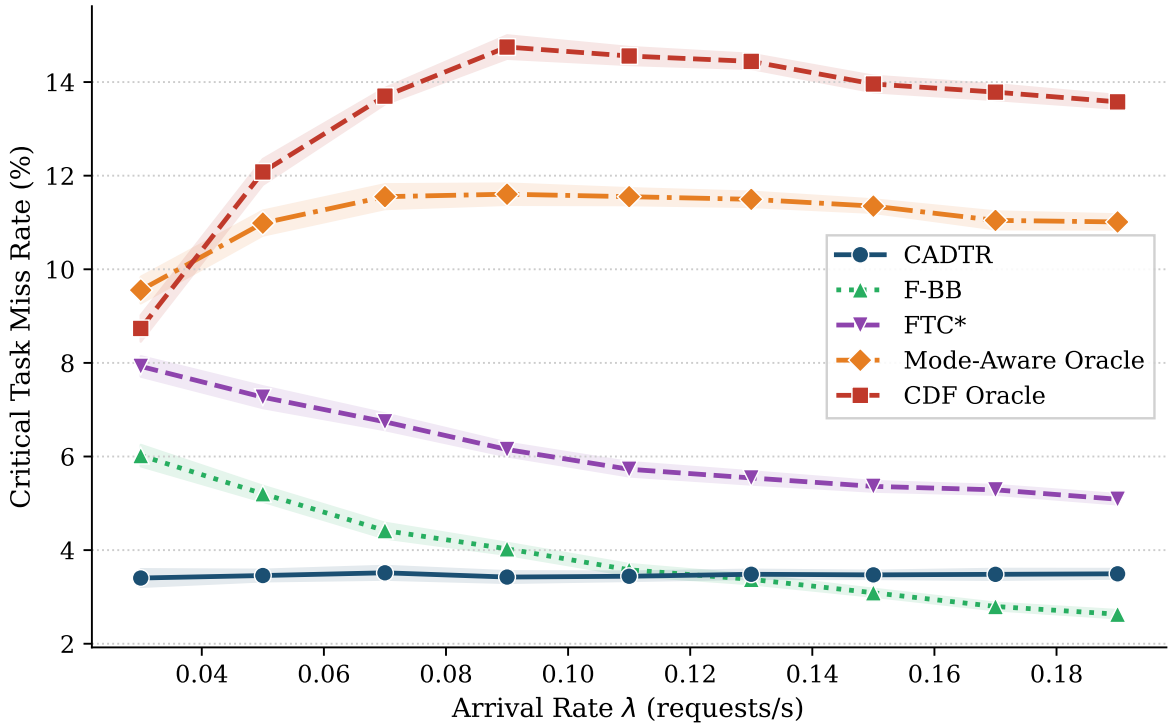

Figure 1. Critical task miss rate vs. arrival rate λ. CADTR maintains a flat $\sim$3.5% miss rate while both oracle baselines degrade to 11–14% under load.

Oracle has the worst aggregate miss rate at high load, confirming that myopic per-task expected-utility optimization fails under congestion.

Table 4 summarizes performance at three representative load levels: low $\lambda = 0.03$, medium $\lambda = 0.11$, and high $\lambda = 0.19$.

Several observations emerge from Table 4:

- **CADTR dominates on critical miss rate**: CADTR achieves the lowest critical miss rate at low and medium load, and near-lowest at high load (3.49% vs. F-BB's 2.63%), while maintaining substantially higher utility than F-BB at low load.
- **Oracle failure is systematic**: Both oracles consistently perform worst on critical tasks, despite having strictly more queue-state information than CADTR. This confirms that the failure is structural, not due to parameter miscalibration.
- **Dynamic throttling intensity scales with load**: CADTR's Strategic usage on critical tasks drops from 41% to 5% as λ increases, demonstrating load-responsive behavior.
- **F-BB trades utility for safety**: F-BB achieves the lowest *overall* miss rate at high load, but at the cost of over-conservative Tactical bias and lower utility at low load. CADTR achieves a better balance.

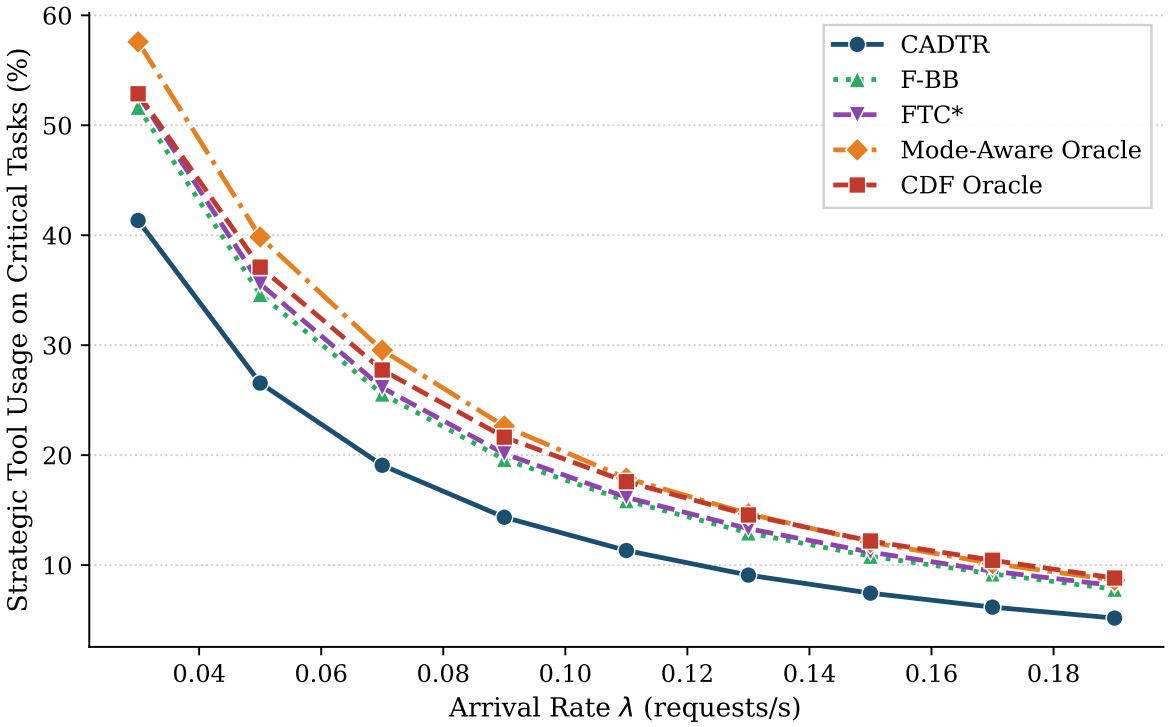

Figure 2. Strategic tool usage on critical tasks vs. λ. CADTR dynamically throttles from 41% to 5%, while oracles remain at $\sim$9% at high load.

7.3 Empirical service-time distributions

Figure 5 plots the empirical ECDF of measured service latencies for the Tactical (*fast*) and Strategic (*slow*) pipelines across the three model backends: Llama 3.1, Qwen 2.5 7B Instruct, and Mistral 7B Instruct. The same qualitative pattern holds in each case: the Strategic profile is shifted to the right and is typically more variable than the corresponding Tactical profile. This cross-model consistency supports the multi-resolution tool abstraction and helps explain why backlog-aware admission control can achieve large reductions in deadline misses without collapsing to an always-Tactical regime.

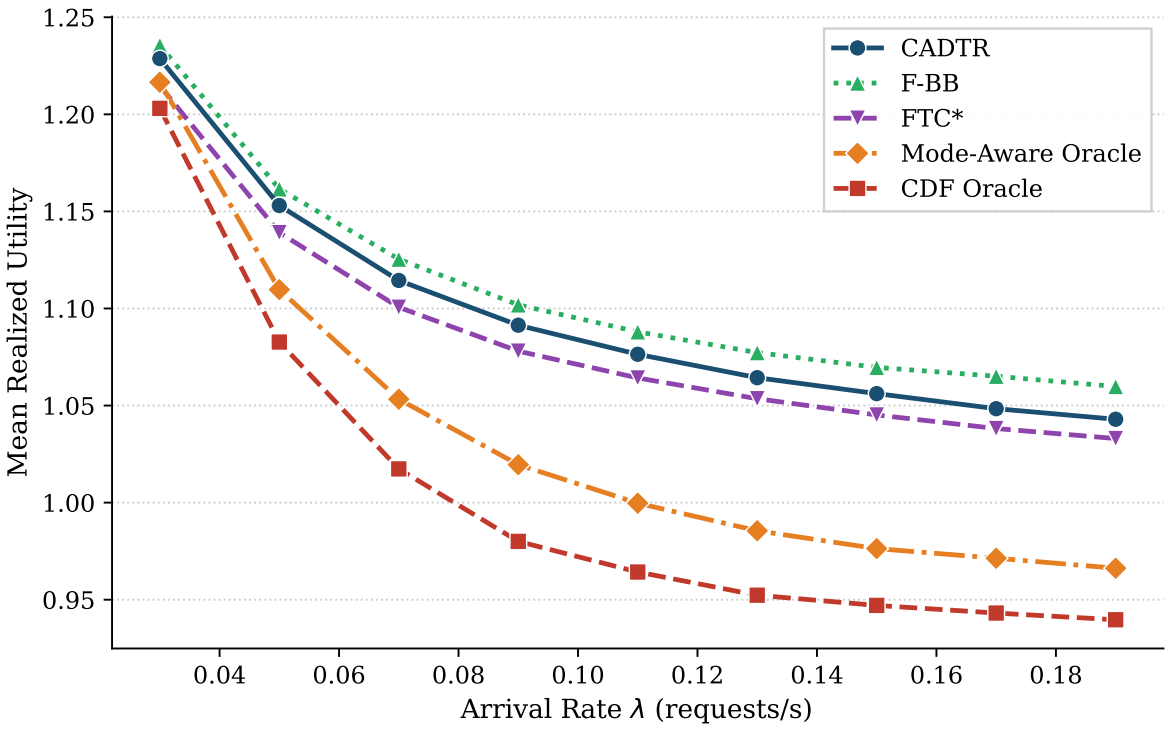

Figure 3. Overall mean realized utility vs. λ. CADTR maintains competitive system utility while providing superior critical-task protection.

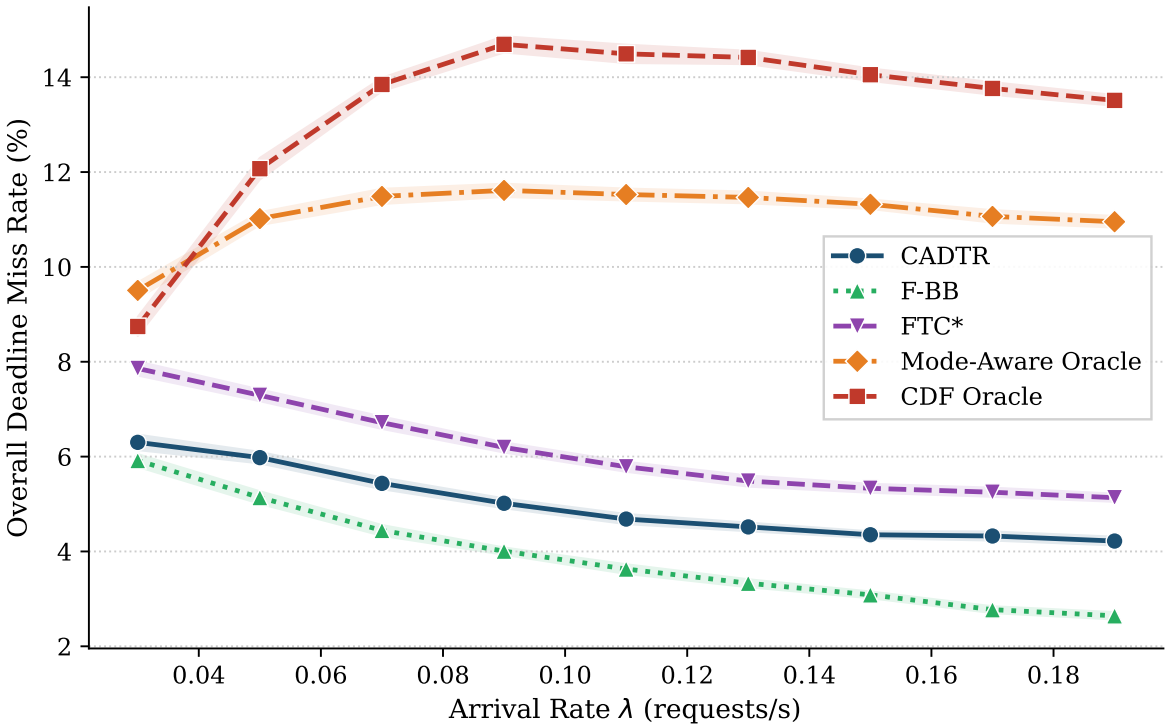

Figure 4. Overall deadline miss rate vs. λ across all task classes.

Table 4. Performance at representative load points ($\lambda \in \{0.03, 0.11, 0.19\}$). Critical miss rate (Crit. MR), overall miss rate (Ovr. MR), overall utility (Ovr. Util.), and Strategic usage fraction on critical tasks (Crit. Strat.%). Best critical MR in **bold**.

λ	Policy	Crit. MR (%)	Ovr. MR (%)	Ovr. Util.	Crit. Strat. (%)
0.03	**CADTR**	**3.40 ± 0.19**	6.30 ± 0.16	1.229 ± 0.002	41.3
	F-BB	6.02 ± 0.23	5.92 ± 0.13	1.236 ± 0.002	51.6
	FTC*	7.93 ± 0.22	7.86 ± 0.15	1.215 ± 0.002	52.8
	Mode-Aware	9.55 ± 0.28	9.50 ± 0.16	1.217 ± 0.003	57.6
	CDF Oracle	8.74 ± 0.28	8.74 ± 0.20	1.203 ± 0.003	52.9
0.11	**CADTR**	**3.44 ± 0.10**	4.68 ± 0.10	1.076 ± 0.001	11.3
	F-BB	3.58 ± 0.12	**3.63 ± 0.10**	1.088 ± 0.001	15.9
	FTC*	5.73 ± 0.15	5.79 ± 0.10	1.064 ± 0.001	16.2
	Mode-Aware	11.55 ± 0.18	11.53 ± 0.12	1.000 ± 0.002	17.9
	CDF Oracle	14.56 ± 0.19	14.49 ± 0.19	0.964 ± 0.002	17.6
0.19	**CADTR**	**3.49 ± 0.10**	4.22 ± 0.09	1.043 ± 0.001	5.2
	F-BB	2.63 ± 0.09	**2.64 ± 0.07**	1.060 ± 0.001	7.8
	FTC*	5.09 ± 0.11	5.13 ± 0.08	1.033 ± 0.001	8.1
	Mode-Aware	11.01 ± 0.16	10.95 ± 0.12	0.966 ± 0.001	8.6
	CDF Oracle	13.58 ± 0.15	13.51 ± 0.11	0.940 ± 0.001	8.8

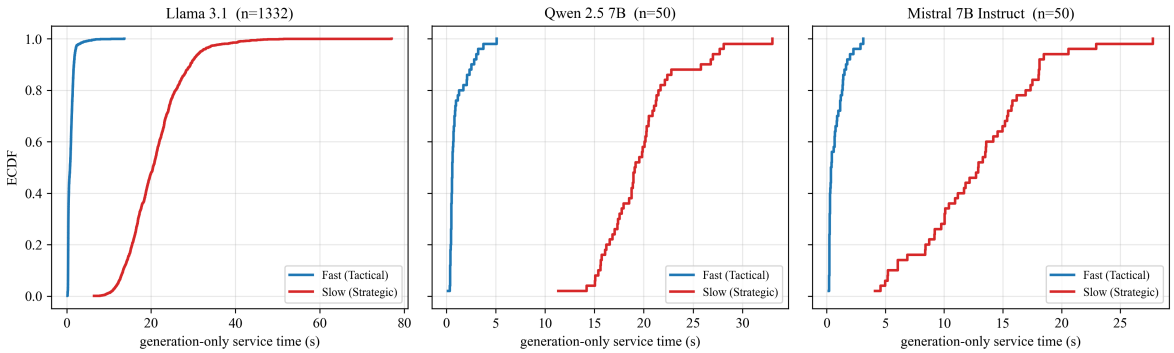

Figure 5. Empirical ECDF of service times for Tactical (FAST) and Strategic (SLOW) pipelines across three counterfactually measured model backends. In each panel, the Strategic pipeline is stochastically slower, supporting the multi-resolution tool abstraction.

8 DISCUSSION

Both oracle baselines possess strictly more quantitative queue information than CADTR, yet they miss far more critical tasks under load. The Mode-Aware Oracle computes exact per-task waiting times by inspecting every queued task's assigned mode, eliminating the approximation error of the ρ-based backlog estimator, and the CDF Oracle maximizes expected on-time utility via a Gaussian CDF approximation. Both treat critical and standard tasks identically, so under congestion they accept the same level of risk for a routine status update as for a mission-critical alert. CADTR breaks this symmetry with a single additive ε-shift, one conditional branch in Algorithm 2, that proactively routes critical tasks to the Tactical tool under congestion. The lesson is that semantic awareness can outperform purely quantitative optimization when the cost of failure is asymmetric across task classes.

The same mechanism acts as a load-adaptive throttle on Strategic-tool usage (Figure 2). At low load, critical tasks use the Strategic tool 41% of the time; as congestion grows, CADTR progressively restricts Strategic usage to 5%, reserving the deadline-feasibility budget for timely completion. This behavior follows from the ε-shift interacting with the monotone threshold structure of Theorem 4.1: a tighter ε contracts the Strategic-tool admission region precisely when backlog threatens feasibility. Feasibility gating is robust because it couples mode selection to the quantity that defines timeliness, the remaining deadline budget δ_k: when backlog grows, the waiting-time estimate $\widehat{W}_k$ rises, more arrivals are assigned to the Tactical tool, the induced mean service time falls, and queueing delays stabilize. Operationally, the policy exposes two interpretable parameters that can be tuned independently: the time margin ε sets the baseline operating point for standard traffic, while the priority-dependent shift $\Delta_\varepsilon^{\text{crit}}$ sets the safety margin for critical events.

8.1 Limitations and extensions

This paper focuses on the single-server setting to isolate the feasibility-gating principle in the simplest nontrivial queue where backlog, deadlines, and mode choice interact transparently. Multi-server, multi-GPU, or batched extensions are important future directions. The binary critical/standard classification could be generalized to a continuous priority spectrum with graded ε-shifts. Online calibration of $\hat{s}(\cdot)$ and $\Delta_\varepsilon^{\text{crit}}$ via feedback control is another natural extension. Finally, CADTR composes with token-level and kernel-level serving optimizations by acting at an orthogonal layer: it shapes the workload by selecting among tool-permission profiles, while serving kernels optimize execution of the chosen work.

9 CONCLUSION

We studied deadline-constrained tool orchestration for autonomous agents through a queueing model with non-preemptive service, firm deadlines, and a multi-resolution utility structure. In this setting, tool permissioning is a systems-level control problem: granting access to the slower Strategic tool improves per-task decision quality, but it also increases congestion and the risk of

missed deadlines.

We introduced *Firm-Deadline Tool Control (FTC)*, an arrival-epoch feasibility-gating policy with a provably exact monotone threshold structure in estimated backlog, deadline budget, and the time-margin parameter ε, including a generalization to arbitrarily many ordered tool tiers. We then introduced *Context-Aware Dynamic Tool Resolution (CADTR)*, which extends FTC with a minimal but consequential mechanism: event-driven dynamic risk adaptation via a priority-dependent ε-shift.

The central empirical finding is that CADTR's semantic awareness—the ability to distinguish "this is a critical event" from "this is a routine task"—proves more valuable than the mathematical sophistication of oracle baselines that possess strictly more quantitative information. Under a nine-point load sweep with 25% critical tasks, both a Mode-Aware Oracle (exact per-task queue-wait knowledge) and a Myopic CDF Oracle (expected-utility maximization) drop up to $\sim$14% of critical tasks, while CADTR maintains a practically flat $\sim$3.5% critical miss rate by dynamically throttling Strategic tool usage from 41% down to 5% as load increases.

This result suggests a broader principle for the design of mission-critical AI systems: when the cost of failure varies across event types, a lightweight semantic signal (event priority) can outperform quantitatively superior but semantically blind optimization. CADTR demonstrates this principle in the simplest nontrivial setting; extending it to multi-server architectures, continuous priority spectra, and real-time deployed autonomous agents is future work.

CODE AND DATA AVAILABILITY

The complete source code, empirical LLM trace data, and the pipeline that reproduces every figure and table in this article are archived at Zenodo (DOI: `https://doi.org/10.5281/zenodo.21936394`). The full development history is publicly available at `https://github.com/maliozturk/CADTR` and `https://github.com/maliozturk/Deadline_Aware_Tool_Permissioning`.

ACKNOWLEDGEMENTS

The core ideas and all scientific content of this article were authored by the authors. After preparing the initial draft, the authors used AI-based writing assistance—primarily Google Gemini 3.5 Flash (accessed via Google Antigravity), with OpenAI Codex 5.4—to correct grammar and improve the academic phrasing of the manuscript. The authors reviewed and edited all resulting text and take full responsibility for the content of the article.

DECLARATION OF COMPETING INTEREST

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

CREDIT AUTHORSHIP CONTRIBUTION STATEMENT

Muhammet Ali Öztürk: Conceptualization, Methodology, Software, Validation, Formal analysis, Writing - Original Draft. **Harun Artuner**: Supervision, Project administration, Writing - Review & Editing.

DATA AND CODE AVAILABILITY

The code and data required to reproduce all experiments, tables, and figures—including the simulator, the empirical service-time traces, and the analysis/plotting scripts—are publicly available at `https://github.com/maliozturk/CADTR`.

REFERENCES

Agrawal, A., Kedia, N., Mohan, J., Panwar, A., Kwatra, N., Gulavani, B. S., Ramjee, R., and Tumanov, A. (2024a). Vidur: A large-scale simulation framework for llm inference.

Agrawal, A., Kedia, N., Panwar, A., Mohan, J., Kwatra, N., Gulavani, B. S., Tumanov, A., and Ramjee, R. (2024b). Taming throughput-latency tradeoff in llm inference with sarathi-serve.

Chen, L., Zaharia, M., and Zou, J. (2023). Frugalgpt: How to use large language models while reducing cost and improving performance.

Chen, S., Jia, Z., Khan, S., Krishnamurthy, A., and Gibbons, P. B. (2025). SLOs-Serve: Optimized serving of multi-SLO LLMs.

Gujarati, A., Karimi, R., Alzayat, S., Hao, W., Kaufmann, A., Vigfusson, Y., and Mace, J. (2020). Serving DNNs like clockwork: Performance predictability from the bottom up. In *14th USENIX Symposium on Operating Systems Design and Implementation (OSDI 20)*, pages 443–462. USENIX Association.

Hellerstein, J. L., Diao, Y., Parekh, S., and Tilbury, D. M. (2004). *Feedback Control of Computing Systems*. Wiley-IEEE Press.

Hong, K., Li, X., Chen, L., Mao, Q., Dai, G., Ning, X., Yan, S., Liang, Y., and Wang, Y. (2025). SOLA: Optimizing SLO attainment for large language model serving with state-aware scheduling. MLSys 2025 (Poster).

Ibrahim, R. and Whitt, W. (2009). Real-time delay estimation based on delay history. *Manufacturing & Service Operations Management*, 11(3):397–415.

Kluge, F., Neuerburg, M., and Ungerer, T. (2015). Utility-based scheduling of $$(m,k)$$-firm real-time task sets. In Pinho, L. M. P., Karl, W., Cohen, A., and Brinkschulte, U., editors, *Architecture of Computing Systems – ARCS 2015*, pages 201–211, Cham. Springer International Publishing.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. (2023). Efficient memory management for large language model serving with PagedAttention. In *Proceedings of the 29th ACM Symposium on Operating Systems Principles (SOSP)*. Also available as arXiv:2309.06180.

Liu, C. L. and Layland, J. W. (1973). Scheduling algorithms for multiprogramming in a hard-real-time environment. *Journal of the ACM*, 20(1):46–61.

López, J. M., Díaz, J. L., and García, D. F. (2004). Utilization bounds for EDF scheduling on real-time multiprocessor systems. *Real-Time Systems*, 28(1):39–68.

Lu, C., Stankovic, J. A., Son, S. H., and Tao, G. (2002). Feedback control real-time scheduling: Framework, modeling, and algorithms. *Real-Time Systems*, 23(1–2):85–126.

Luo, M. (2025). Autellix: An efficient serving engine for llm agents as general programs.

Neely, M. J. (2010). *Stochastic Network Optimization with Application to Communication and Queueing Systems*. Morgan & Claypool Publishers.

Nie, C., Fonseca, R., and Liu, Z. (2024). Aladdin: Joint placement and scaling for SLO-aware LLM serving.

O. Garnett, A. Mandelbaum, M. R. (2002). Designing a call center with impatient customers. In *Manufacturing & Service Operations Management 4(3):208–227*.

Ong, I., Almahairi, A., Wu, V., Chiang, W.-L., Wu, T., Gonzalez, J. E., Kadous, M. W., and Stoica, I. (2024). Routellm: Learning to route llms with preference data.

Patke, A., Reddy, D., Jha, S., Pinto, C., Qiu, H., Cui, S., Narayanaswami, C., Kalbarczyk, Z. T., and Iyer, R. K. (2024). Queue management for large language model serving. In *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS)*. Also circulated as arXiv:2407.00047.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. (2023). Toolformer: Language models can teach themselves to use tools.

Stankovic, J. A., Lu, C., Son, S. H., and Tao, G. (1999). The case for feedback control real-time scheduling. In *Proceedings of the 11th Euromicro Conference on Real-Time Systems (ECRTS)*, pages 11–20.

Sun, B. et al. (2024). Llumnix: Dynamic scheduling for large language model serving.

Tang, Y., Lan, T., Huang, X., Lu, H., and Chen, W. (2025). Scorpio: Serving the right requests at the right time for heterogeneous slos in llm inference.

Tao, Y. (2025). Prompt-aware scheduling for low-latency llm serving.

Wang, X., Liu, Y., Cheng, W., Zhao, X., Chen, Z., Yu, W., Fu, Y., and Chen, H. (2025). MixLLM: Dynamic routing in mixed large language models. In Chiruzzo, L., Ritter, A., and Wang, L., editors, *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, pages 10912–10922, Albuquerque, New Mexico. Association for Computational Linguistics.

Wang, Y. (2025a). Astraea: A state-aware scheduling engine for llm-powered agents.

Wang, Y. (2025b). Augserve: Adaptive request scheduling for augmented large language model inference services.

Wei, X. (2025). Agent.xpu: Efficient scheduling of agentic llm workloads on heterogeneous socs.

Whitt, W. (2006). Fluid models for multiserver queues with abandonments. *Operations Research*, 54(1):37–54.

Wu, B., Zhong, Y., Zhang, Z., Huang, G., Liu, X., and Jin, X. (2023). Fast distributed inference serving for large language models.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. (2023). React: Synergizing reasoning and acting in language models.

Zhang, H., Tang, Y., Khandelwal, A., and Stoica, I. (2023). SHEPHERD: Serving DNNs in the wild. In *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI 23)*, pages 787–808, Boston, MA. USENIX Association.

Zhang, W. (2025). Jitserve: Slo-aware llm serving with imprecise request information.

Zhong, Y., Liu, S., Chen, J., Hu, J., Zhu, Y., Liu, X., Jin, X., and Zhang, H. (2024). Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving.